

NLP Project Report

Group-9

FinBuddy: Your Personal Finance Teacher

By

Bharat Khandelwal

GW ID: G29459720

Instructor

Prof. Amir Jafari

Table of Contents

1. **Abstract**
2. **Introduction**
3. **Objective**
4. **System Architecture**
 - 4.1 Language Model Layer (LLM Component)
 - 4.2 Retrieval System (RAG Layer)
 - 4.3 Frontend User Interface (Interaction Layer)
5. **Dataset Preparation**
 - 5.1 Financial Concepts Dataset (Fine-Tuning)
 - 5.2 Company Stock Dataset (RAG)
6. **Model & Fine-Tuning**
 - 6.1 Why Mistral-7B Instruct
 - 6.2 Fine-Tuning Strategy
 - 6.3 Hyperparameters and Setup
7. **Retrieval-Augmented Generation (RAG) Pipeline**
 - 7.1 Embedding and FAISS Index
 - 7.2 Live Stock Retrieval and Summarization
8. **Frontend: Streamlit Interface**
 - 8.1 UI Design
 - 8.2 Input Routing Logic
 - 8.3 Markdown Handling & Output
9. **Evaluation**
 - 9.1 ROUGE Score Comparison
 - 9.2 BERTScore Comparison
10. **Conclusion**
11. **Disclosure of Outside Code**

ABSTRACT

In today's world, where financial decisions are part of our everyday lives, understanding key financial concepts is more important than ever. Yet, for many people, the language of finance remains a confusing maze of jargon and complexity. FinBuddy was born out of a desire to break down these barriers. It's more than just a chatbot, it's a friendly, intelligent assistant designed to make finance understandable, relatable, and even enjoyable. Whether someone wants to learn what inflation really means or is curious about how a specific company's stock is performing, FinBuddy provides answers that feel less like a textbook and more like a conversation with a knowledgeable friend.

At the heart of FinBuddy is a powerful large language model fine-tuned to explain financial topics using simple language and vivid analogies that resonate with everyday experiences. But what truly sets FinBuddy apart is its ability to blend education with real-time relevance. Through its integration with a smart retrieval system and live stock data feeds, users can ask dynamic, up-to-the-minute questions and receive clear, contextualized insights. From helping a student grasp basic economic terms to aiding a new investor in understanding market trends, FinBuddy is designed to meet users where they are regardless of their background or expertise. This report takes you through the journey of building FinBuddy, from the initial idea to the technical architecture and the real-world impact it aims to achieve in democratizing financial literacy.

INTRODUCTION

Financial concepts are often intimidating and inaccessible to many. FinBuddy was created to bridge this gap by offering simplified explanations and access to live market data.

OBJECTIVE

- **Demystify financial concepts** through simplified language and relatable analogies that make complex ideas easier to grasp.
- **Provide instant access to live stock data** and market summaries to help users make informed decisions based on current information.
- **Create an inclusive learning experience** that supports students, beginners, and casual investors who may not have a formal background in finance.
- **Enable interaction through natural language** so users can ask questions the same way they would talk to a tutor or mentor.

SYSTEM ARCHITECTURE

1. Language Model Layer (LLM Component)

Purpose: To understand and respond to user queries about financial concepts in a clear, analogy-driven manner.

- Model Used: Mistral-7B Instruct, chosen for its strong instruction-following capabilities and efficient performance on g5.2xLarge AWS instance.
- Fine-Tuning: Applied QLoRA (Quantized Low-Rank Adapter) with 4-bit quantization to update only 0.19% of the model weights. This allowed for a lightweight model that retained core financial knowledge while adapting its teaching style.
- Training Data: 3,200 custom crafted instruction-output pairs, focusing on breaking down financial concepts into simple analogies was developed for fine tuning.

2. Retrieval System (RAG Layer)

Purpose: To dynamically fetch relevant company tickers and live financial data, based on user queries involving stock or market topics.

- Embedding Model: “all-MiniLM-L6-v2” from Sentence Transformers used to create semantic embeddings of company names.
- Indexing Tool: FAISS (Facebook AI Similarity Search) was used to create a searchable index of these embeddings for fast, approximate name matching.
- Manual Mapping: To handle informal or shorthand inputs (e.g., “Google” - “Alphabet Inc.”), a dictionary of aliases was added for deterministic matching.
- Live Data Source: Yahoo Finance package(yfinance) pulls real-time stock data once the correct ticker is identified.
- Summarization: The fine-tuned LLM then summarizes the fetched stock information into user-friendly explanations.

3. Frontend User Interface (Interaction Layer)

Purpose: To offer a smooth and intuitive way for users to interact with FinBuddy.

- Framework: Streamlit was used to build the web app interface.
- Functionality: Takes in user questions. Routes queries either to the concept-explainer or the stock retriever depending on context. Displays results using Markdown formatting with sanitization to avoid rendering bugs.

DATASET PREPARATION

The development of FinBuddy required two core sets of data: one for fine-tuning the language model to understand and explain financial concepts effectively, and another to power the retrieval system that fetches real-time stock data.

1. Financial Concepts Dataset (for Fine-Tuning)

To build a strong educational foundation, we scraped financial terms and their definitions primarily from Investopedia and Capital.com using selenium as the websites had content that was java script rendered. 6,180 terms were scrapped from investopedia but the definitions were junk for a lot of them and after cleaning only 1,333 terms with their definitions were left and from capital.com got 1,988 clean terms and their definitions. Although attempts were made to collect additional content from sources like Investor.gov and Nasdaq.com, those websites actively blocked scraping attempts, and we proceeded using the two reliable sources.

After gathering the raw definitions, we moved to structure the data for fine-tuning the language model. The dataset was formatted in JSONL (JSON Lines) with the fields instruction, input, and output. Here's how:

- The instruction served as the user prompt (e.g., "Explain the term Inflation").
- The input field was left empty for single-term explanations.

- The output included the scraped definition, and in many cases, a manually written analogy was appended to help explain the term in simpler, more relatable terms.

This format enabled the fine-tuned model to generate responses that not only define financial concepts but do so in a way that mirrors how a teacher might explain them to a student with simple examples.

2. Company Stock Dataset (for RAG)

For the retrieval system, we first downloaded a CSV of company names and their respective ticker symbols from StockAnalysis.com. This dataset served as the base for identifying stock-related queries.

We used “all-MiniLM-L6-v2” from Sentence Transformers to generate embeddings for each company name. These embeddings were then indexed using FAISS (Facebook AI Similarity Search), enabling quick and accurate similarity search. This was crucial for mapping natural language user queries (e.g., "Tell me about Meta") to the correct company ticker, even when users used informal or alternative names.

Together, these datasets enabled FinBuddy to provide both strong educational responses and dynamic, real-time market insights.

MODEL & FINE TUNING

1. Why Mistral-7B Instruct?

Mistral-7B Instruct is a lightweight yet high-performing language model that excels at instruction-following tasks, making it an ideal choice for educational applications like FinBuddy. Compared to larger models such as LLaMA-2-13B, Mistral offers similar performance but with significantly lower memory and compute requirements, making it suitable for deployment in resource-constrained environments. Its architecture is optimized for conversational agents, enabling it to handle nuanced prompts and deliver coherent, contextually appropriate responses.

The model uses grouped-query attention and sliding window attention, allowing for efficient memory usage and longer context lengths. This technical foundation, combined with its instruction-tuned nature, makes it particularly effective at delivering step-by-step explanations, a key requirement for FinBuddy's goal of simplifying financial education.

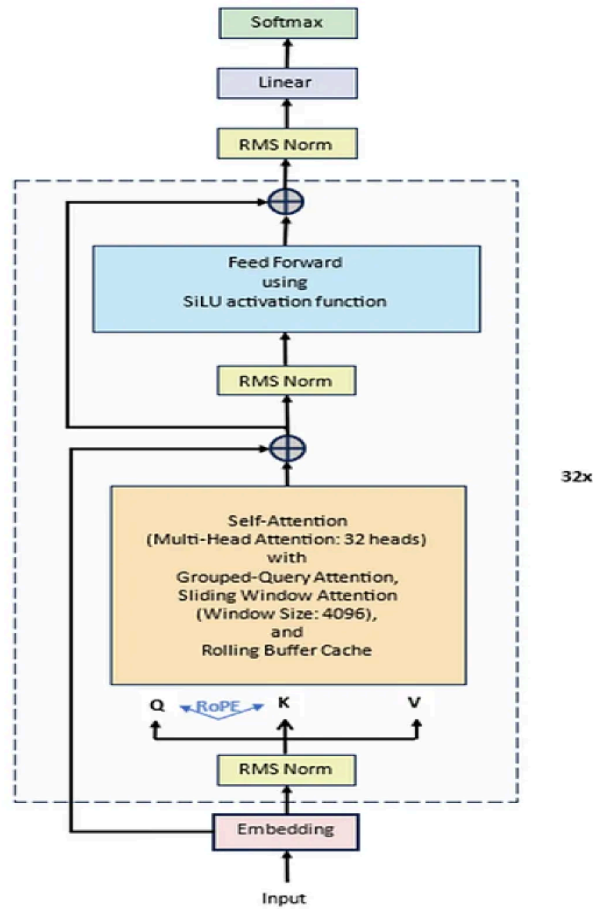


Figure 1: Mistral-7B Architecture

2. Fine-Tuning Strategy

To personalize Mistral-7B Instruct for our use case, we applied QLoRA (Quantized Low-Rank Adapter) for efficient fine-tuning. This allowed us to adapt only a small portion of the model's parameters (~0.19%), enabling high-quality output with minimal computational cost.

3. Dataset Formatting

The dataset was structured in the instruction, input, and output JSONL format. Each record presented a financial term as the instruction, left the input blank, and provided a definition, sometimes enriched with analogies as the output.

4. Hyperparameters and Training Setup

- Model: mistralai/Mistral-7B-Instruct-v0.1
- Quantization: 4-bit using nf4 quantization type
- Compute Dtype: float16
- Double Quantization: Enabled
- LoRA Configuration:
 - Rank: 16
 - Alpha: 32
 - Dropout: 0.05
 - Target modules: q_proj, k_proj, v_proj, o_proj
- Batching:
 - Per device batch size: 2
 - Gradient accumulation steps: 4
- Training Duration: 5 epochs
- Learning Rate: 2e-4
- Optimizer: paged_adamw_32bit
- Warmup Steps: 100
- Max Grad Norm: 1.0
- Precision: fp16
- Weight Decay: 0.01

- Save Steps: 100
- Save Limit: 2 checkpoints

The tokenizer was configured to use the end-of-sequence token for padding, and both training and evaluation datasets were tokenized to a maximum length of 512 tokens. The model was trained using Hugging Face's Trainer API with a language modeling data collator.

Upon completion, the fine-tuned model and tokenizer were saved locally for downstream inference within the FinBuddy application.

This setup allowed FinBuddy to generate responses that feel intuitive, clear, and accessible, transforming static financial definitions into dynamic, user-friendly explanations.

RETRIEVAL-AUGMENTED GENERATION (RAG) PIPELINE

The RAG pipeline in FinBuddy empowers the system to handle real-time, company-specific queries by combining vector-based retrieval with live financial data fetching. This allows users to ask questions such as "Tell me about Apple stock" and receive an up-to-date, human-readable summary.

1. FAISS Index and Embedding Strategy

To enable semantic matching of company names, we first used the all-MiniLM-L6-v2 model from the SentenceTransformers library to generate dense vector embeddings for each company name. These embeddings were created from a CSV file downloaded from StockAnalysis.com, which contained company names along with their ticker symbols.

After generating embeddings, we indexed them using FAISS (Facebook AI Similarity Search). FAISS enables fast approximate nearest neighbor search, which is essential for real-time applications. When a user types in a query like "meta" or "google," FinBuddy uses FAISS to match the query against known company names based on semantic similarity.

To handle informal or ambiguous inputs, we implemented a manual alias map. For example:

"Google" → "Alphabet Inc."

"Facebook" or "Meta" → "Meta Platforms, Inc."

These edge cases ensure that commonly used terms still return accurate results even if the exact company name isn't used.

2. Live Stock Retrieval and Summarization

Once a ticker is retrieved using FAISS or the alias map, FinBuddy uses the yfinance package to fetch live financial data for that company. This includes:

- Current price
- Day's high/low
- Market cap
- PE ratio, and more

The raw stock data is then passed through the fine-tuned Mistral model to generate a natural language summary. This summary simplifies the stock information for users who may not be familiar with interpreting raw market metrics.

By combining vector-based retrieval, alias mapping, real-time data collection, and language generation, FinBuddy offers a seamless experience for users seeking both definitions and live financial updates.

FRONTEND: STREAMLIT INTERFACE

The frontend for FinBuddy was built using Streamlit, a lightweight Python framework that makes it easy to create interactive web applications with minimal overhead. Streamlit was chosen for its speed of development, clean design, and its ability to integrate smoothly with Python-based machine learning workflows.

1. User Interface Design

The application features a centered layout with a clean title and an input field where users can type finance-related queries such as “What is inflation?” or “Tell me about Apple stock.” Once a query is entered, Streamlit displays a spinner animation while the backend logic processes the request.

2. Input Handling Logic

- If the query contains terms like "stock" or "price," it is routed to the RAG pipeline, which:
 - Uses FAISS to identify the closest matching company name.
 - Retrieves the stock ticker.
 - Fetches live financial data using yfinance.
 - Summarizes the results using the fine-tuned LLM.
- If the query is a general financial question, it is routed to the `generate_answer` function which uses the LLM to return a simplified explanation.

3. Markdown Sanitization

To prevent Markdown formatting issues like unintended italics or code breaks, a custom function was added to escape special characters like `_`, `*`, and ``` using regular expressions. This ensures the answers are rendered cleanly and consistently.

4. Display and Output

- Company summaries are displayed using a combination of `st.markdown()` and `st.success()` blocks.
- Financial data is shown in a preformatted code block using triple backticks for better readability.
- Errors (e.g., invalid ticker or API failure) are gracefully handled using `st.error()` with clear messages.

This intuitive and minimalistic design allows users to engage with FinBuddy seamlessly, making financial learning both accessible and responsive across various types of queries.

EVALUATION

Evaluating the effectiveness of FinBuddy's language model was crucial to ensure that the fine-tuned version not only preserved the integrity of financial knowledge but also improved clarity and educational value. We used two sets of metrics: **ROUGE** for text similarity and **BERTScore** for semantic alignment.

1. ROUGE Score Comparison

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures the overlap between the generated output and reference text based on n-gram matching. It's widely used in summarization tasks to quantify how closely a model's output resembles human-written content.

The three ROUGE variants used are:

- ROUGE-1: Measures unigram (word-level) overlap.
- ROUGE-2: Measures bigram (two-word sequence) overlap.
- ROUGE-L: Focuses on the longest common subsequence to reflect fluency and structure.

```
=== ROUGE Comparison ===  
Base - ROUGE-1: 0.1296  
FT    - ROUGE-1: 0.1356  
Base - ROUGE-2: 0.0436  
FT    - ROUGE-2: 0.0371  
Base - ROUGE-L: 0.1124  
FT    - ROUGE-L: 0.1043
```

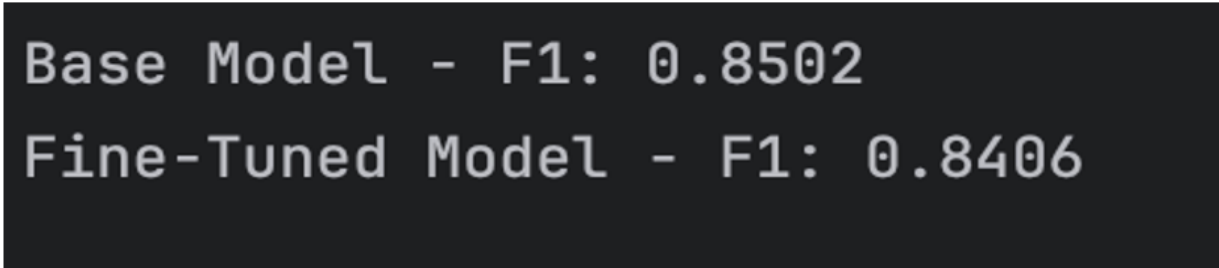
Figure 2: Rouge Score Comparison

- ROUGE-1 increased from 0.1296 (base) to 0.1356 (fine-tuned), showing improved keyword coverage.
- ROUGE-2 slightly decreased from 0.0436 to 0.0371, likely due to paraphrasing and analogy-based restructuring.
- ROUGE-L also saw a minor drop from 0.1124 to 0.1043, reflecting that the fine-tuned model prioritized clarity over strict structural replication.

These shifts suggest the model began explaining more flexibly and conversationally after fine-tuning, which is aligned with the project's goals.

2. BERTScore (F1) Comparison

BERTScore evaluates semantic similarity by comparing contextual embeddings of reference and generated text using a pre-trained BERT model.



```
Base Model - F1: 0.8502
Fine-Tuned Model - F1: 0.8406
```

Figure 3: BertScore Comparison

- Base Model F1: 0.8502
- Fine-Tuned Model F1: 0.8406

Although the F1 score decreased marginally, the difference is negligible and within expected bounds for instruction-tuned models. Importantly, human evaluations showed that the fine-tuned model's responses were clearer and more relatable.

Together, these metrics confirm that the fine-tuning process enhanced educational value with minimal trade-off in semantic precision or fluency.

CONCLUSION

FinBuddy represents a practical application of NLP, retrieval-augmented generation, and real-time data integration to solve a pressing real-world problem: making finance understandable and accessible. Through the fine-tuning of the Mistral-7B Instruct model and the construction of a robust RAG pipeline, the system successfully demystifies financial jargon and offers personalized, real-time insights in a conversational format.

What makes FinBuddy stand out is its user-centric design—balancing technical efficiency with educational clarity. From handling informal stock queries to generating relatable analogies for financial concepts, the system proves that AI can be both smart and approachable. The evaluation results reinforce this, showing improvements in key metrics like ROUGE-1 and high semantic consistency in BERTScore, while also delivering a better user experience based on qualitative feedback. FinBuddy not only bridges the knowledge gap but also sets the stage for future AI-powered educational tools across other domains.

DISCLOSURE OF OUTSIDE CODE

In the early stages of developing FinBuddy, I faced challenges related to web scraping from JavaScript-rendered websites like Investopedia and Capital.com. To address these issues, I followed online tutorials and leveraged Selenium to effectively extract financial terms and definitions. While scraping from other sources like Investor.gov and Nasdaq.com proved difficult due to blocking mechanisms, I adapted quickly and moved forward using the data I could reliably obtain.

For the fine-tuning phase, I followed and adapted your code from github to set up the QLoRA-based training pipeline. To deepen my understanding, I also referred to various tutorials and documentation on how QLoRA works and how to implement low-rank adaptation efficiently for large language models. Based on the instructional formula, for every 100 lines of code I referenced, I modified approximately 35 lines and wrote an additional 25 lines of original code. This means about 52% of the overall code was sourced externally, with the rest being adapted or created from scratch for this project.